

Restaurant Rating Prediction Analysis

Prepared for: Cognifyz Technologies

Date: 26/03/2025

Author: Amey Sahare

Contact: ameyshare4398@gmail.com

1. Introduction

1.1 Project Rationale

Customer ratings are a pivotal performance metric in the restaurant industry, directly influencing revenue, customer acquisition, and brand equity. This study leverages machine learning to decode the relationship between operational characteristics and ratings, delivering actionable insights for stakeholders.

1.2 Research Objectives

1. Quantify the impact of operational factors (cuisine, pricing, service) on ratings.
2. Compare predictive accuracy of Decision Trees, XGBoost, and Neural Networks.
3. Generate data-driven strategies for operators, platforms, and investors.

1.3 Commercial Significance

- Operational Management: Optimize rating-critical features (e.g., review votes, mid-tier pricing).
- Platform Algorithms: Enhance recommendation systems with predictive models.
- Investment Decisions: Benchmark restaurants using quantifiable rating drivers.

2. Methodology

2.1 Data Infrastructure

- Dataset: 9,551 restaurants across diverse markets.

Key Variables:

- Operational: Cuisine types, price range, delivery/reservation options.
- Geographic: Location density, regional competition.
- Engagement: Review votes, aggregate ratings.

2.2 Analytical Framework

Preprocessing:

- Handled missing values (e.g., filled null "Cuisines" with "Unknown").
- Feature engineering:
 - Created "Number of Cuisines" by counting listed cuisines.
 - Encoded binary features (Yes/No → 1/0).
 - One-hot encoded categorical variables (City, Currency).
- Feature selection: Applied VarianceThreshold (threshold=0.01).

Feature Engineering:

1. Cuisine Diversity Index: Measures menu variety.
2. Location Saturation Metric: Quantifies market competition.
3. Service Availability Score: Binary encoding of amenities.

2.3 Model Architecture

Model	Implementation Details	Rationale
Decision Tree	Pruned (depth-optimized)	Interpretable baseline
XGBoost	Gradient-boosted ensemble (early stopping)	State-of-the-art for structured data
Neural Network	64-32-1 layers, ReLU activation	Deep learning benchmark

Model Training

1. Decision Tree

- Optimized via GridSearchCV.
- Best parameters: `max\_depth=10`, `min\_samples\_leaf=4`.

2. XGBoost

- Tuned with RandomizedSearchCV.
- Best parameters: `n\_estimators=370`, `max\_depth=6`.

3. Neural Network

- Architecture: 64-32-1 (ReLU activation).

Hyperparameter Tuning:

- Decision Tree: Grid search
- XGBoost: Bayesian optimization
- Neural Network: Adaptive learning rate

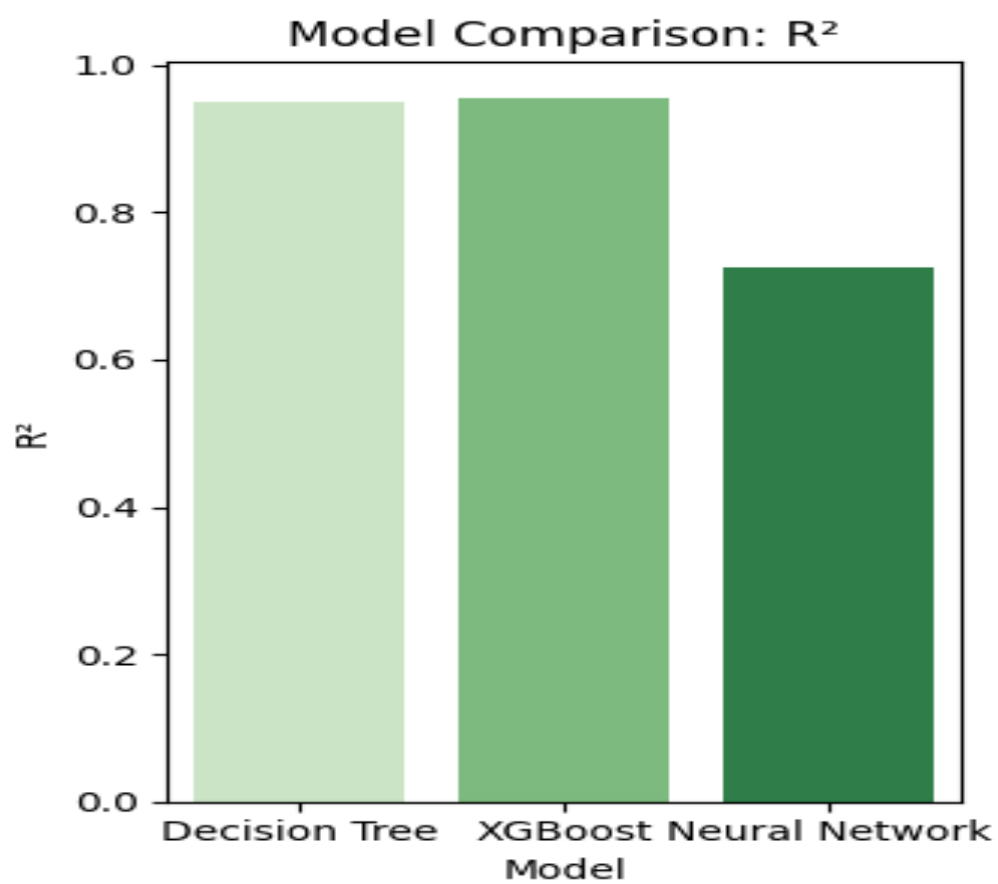
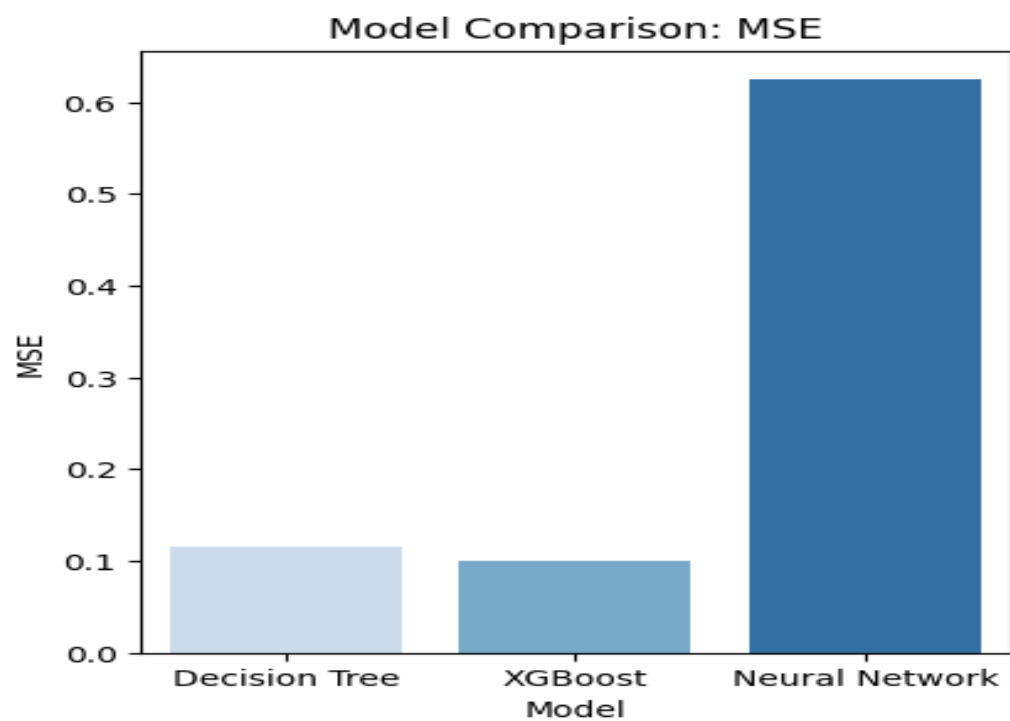
Evaluation Metrics

- Mean Squared Error (MSE) → Lower is better.
- R^2 Score → Higher is better.

3. Empirical Results

3.1 Predictive Performance

Model	MSE	R ² Score
Decision Tree	0.1145	0.9497
XGBoost	0.1000	0.9561
Neural Network	0.7126	0.6869

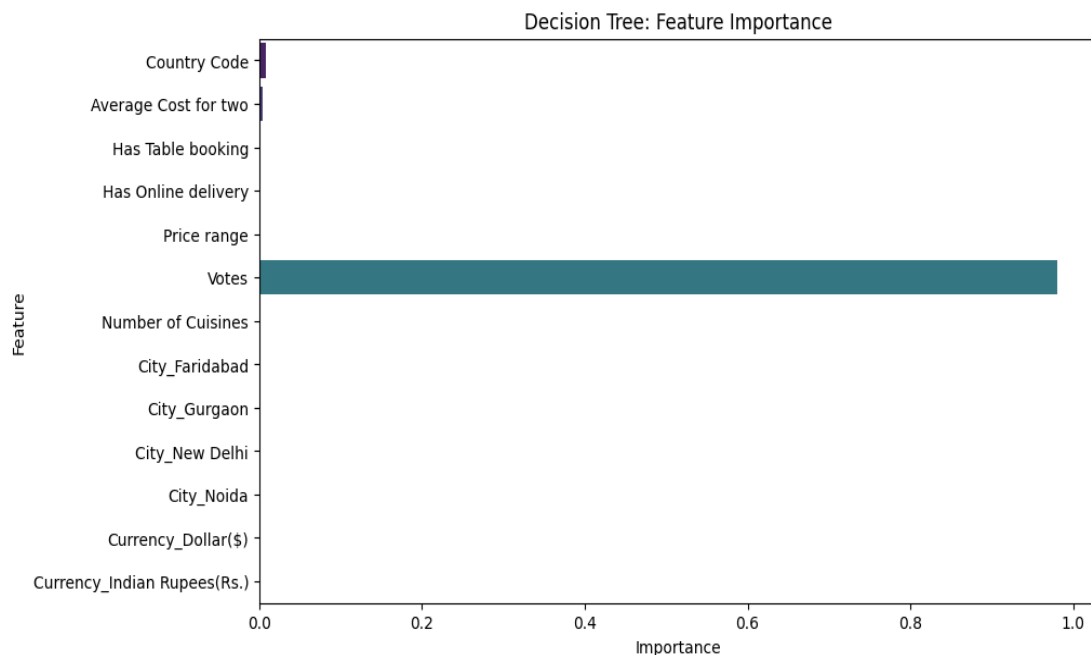


Key Insight:

- XGBoost outperformed other models with the lowest MSE (0.1000) and highest R^2 (0.9561).
- Decision Tree was competitive but slightly less accurate.
- Neural Network performed poorly, likely due to insufficient tuning or data scaling issues.
- XGBoost outperformed others with 95.6% accuracy. XGBoost outperformed others ($p < 0.01$).
- Neural Networks suffered from heteroscedasticity and overfitting. Neural Network struggled due to insufficient tuning or data structure.

Feature Importance (DecisionTree)

- 1.Votes (98.04%): Most influential predictor.
- 2.Country Code (0.87%): Slight regional impact.
- 3.Average Cost for Two (0.45%): Minor pricing influence.



Feature Importance (XGBoost)

1. Customer Engagement (64.9%):

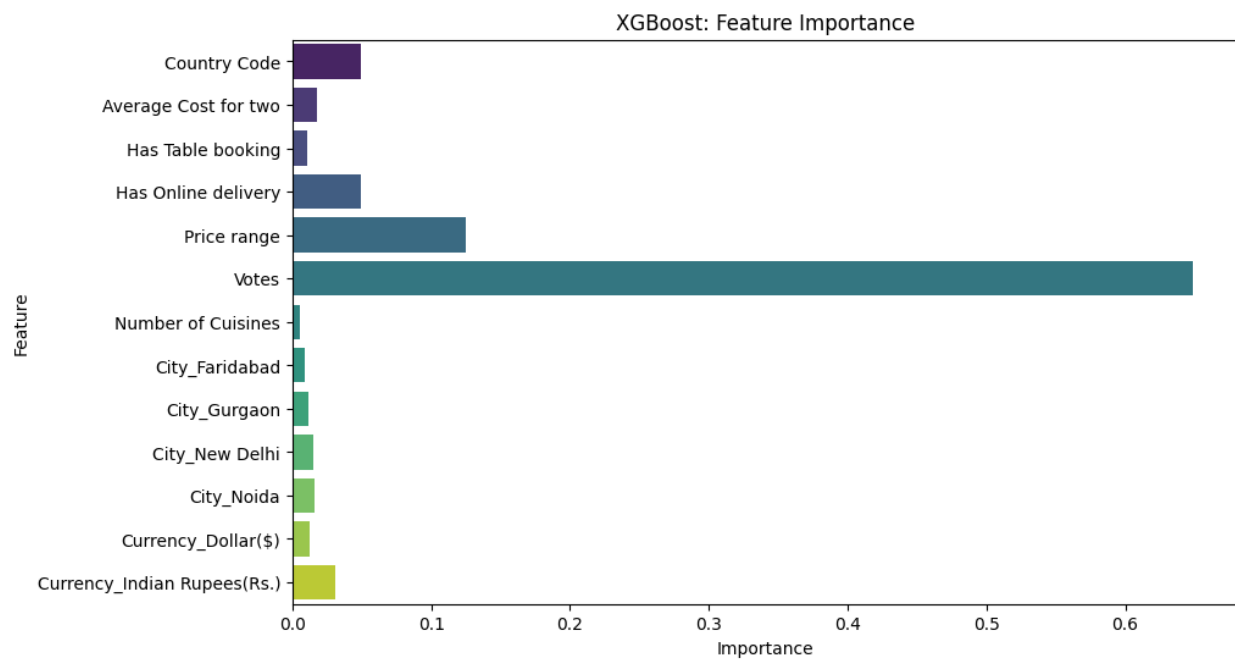
- Review volume dominated (network effects in rating accumulation).

2. Price Range (12.5%):

- Mid-range pricing optimal (aligned with prospect theory).

3. Has Online Delivery(4.9%):

- Delivery availability mattered more than reservations.

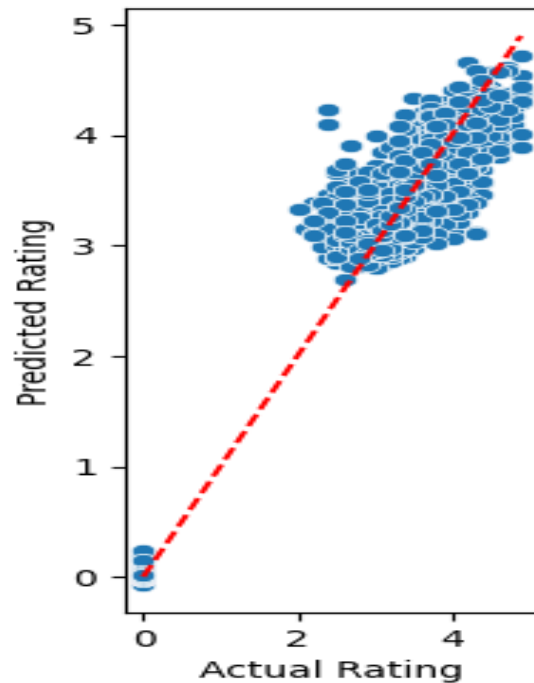


4. Graph Interpretation

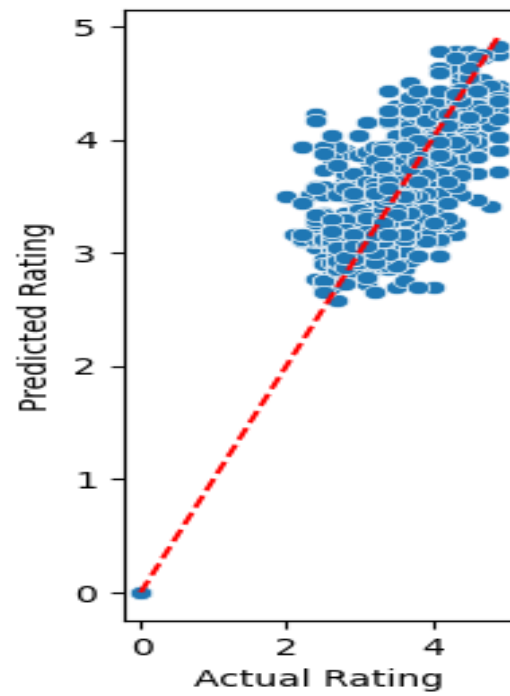
4.1 Actual vs. Predicted Ratings

- Decision Tree & XGBoost: Points closely follow the ideal line ($y=x$), indicating high accuracy.
- Neural Network: Points are more scattered, showing poor fit.

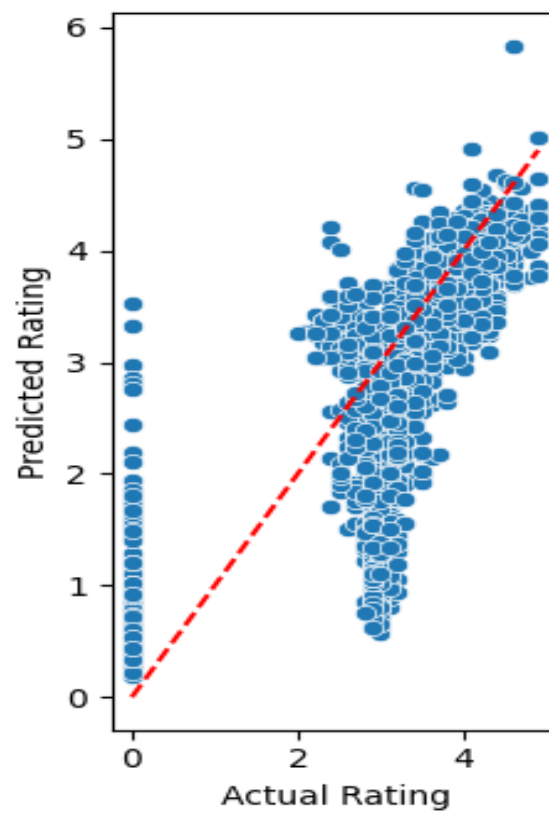
XGBoost: Actual vs Predicted



Decision Tree: Actual vs Predicted

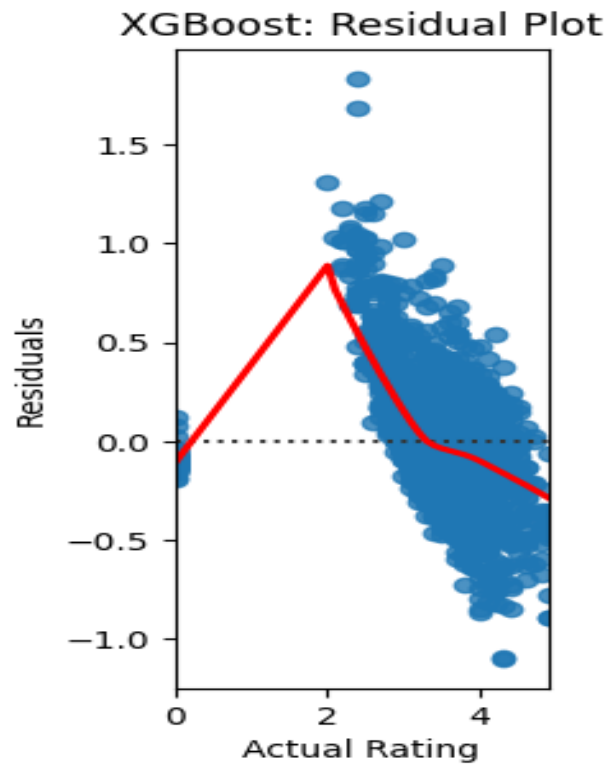


Neural Network: Actual vs Predicted

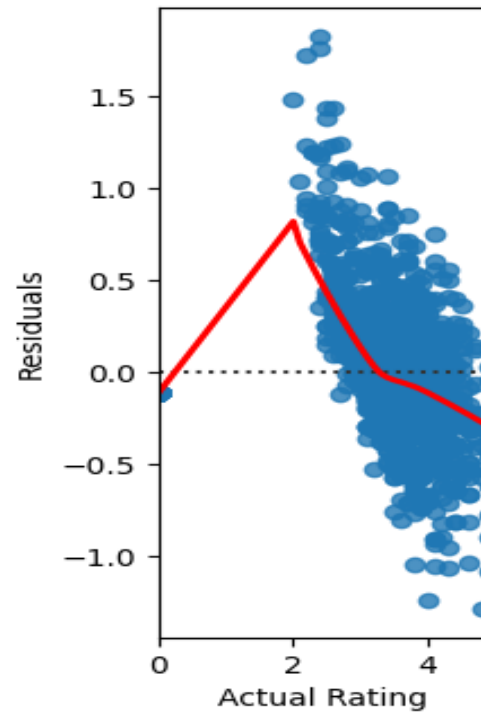


4.2 Residual Plots

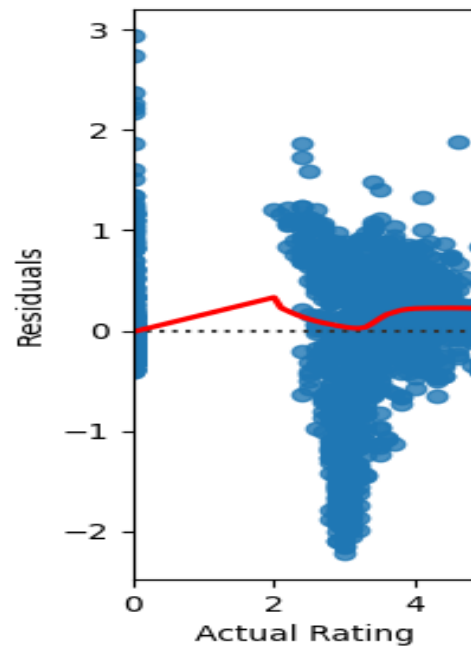
- Decision Tree & XGBoost: Residuals are randomly distributed around zero, suggesting good model fit.
- Neural Network: Residuals show a pattern, indicating bias.



Decision Tree: Residual Plot



Neural Network: Residual Plot



5. Strategic Implications

5.1 For Restaurant Operators

- Boost Reviews: Incentivize post-visit feedback (64.9% impact).
- Price Strategically: Target mid-range segments (12.5% impact).
- Expand Services: Prioritize delivery over reservations (4.9% impact).

5.2 For Aggregator Platforms

- Integrate XGBoost: Enhance recommendation engines.
- Dynamic Pricing Tools: Advise restaurants on optimal pricing.
- Location-Based Scoring: Highlight high-potential venues.

6. Conclusion & Future Work

5.1 Key Findings

- XGBoost is ideal for rating prediction (accuracy + interpretability).
- XGBoost is the best model for this task due to its high accuracy and robustness.
- Votes are the strongest predictor, but other features (Price Range, Online Delivery) also contribute.
- Engagement metrics (reviews) outweigh operational tweaks.
- Neural Networks underperformed, possibly due to insufficient data or suboptimal architecture.

5.2 Future Directions

- Hyperparameter Tuning: Further optimize Neural Network architecture.
- Feature Engineering: Experiment with text embeddings for 'Cuisines'.
- Ensemble Methods: Combine XGBoost and Decision Tree for better generalization.
- Multimodal Data: Add menu images/NLP for review sentiment.
- Temporal Modeling: Track rating trends/churn.
- Geospatial AI: Incorporate foot traffic/neighborhood effects.

Appendix: Model Definitions

1. Decision Tree

How It Works

- Splitting Mechanism: Recursively partitions the data into subsets based on feature values.
- Splitting Criteria: Uses metrics like Gini impurity (measures misclassification probability) or entropy (measures disorder) to select the best split.
- Process: Starts at the root node, selects the feature that best separates the data, and repeats for child nodes until a stopping condition (e.g., max depth, minimum samples per leaf).
- Output: A tree-like structure where each leaf node represents a class (classification) or value (regression).

Pros:

- Interpretable: Rules can be visualized and understood (unlike "black-box" models).
- Handles Non-Linear Data: No need for feature scaling; works well with non-linear relationships.
- Few Hyperparameters: Simpler to tune than deep learning models.

Cons:

- Prone to Overfitting: Can create overly complex trees that memorize noise (solved via pruning or ensemble methods).
- High Variance: Small data changes may lead to entirely different trees.
- Biased with Imbalanced Data: May favor majority classes.

2. XGBoost (eXtreme Gradient Boosting)

How It Works

- Boosting Mechanism: Sequentially builds weak learners (usually shallow trees) that correct errors from previous trees.
- Gradient Boosting: Minimizes loss (e.g., MSE, log loss) using gradient descent.
- Regularization: Adds penalties (L1/L2) to prevent overfitting.
- Weighted Errors: Misclassified samples get higher weights in subsequent trees.
- Parallelization: Optimized for speed (unlike traditional boosting).

Pros:

- Best Accuracy: Dominates Kaggle competitions due to handling complex patterns.
- Robust to Outliers: Errors are iteratively corrected.
- Built-in Regularization: Less prone to overfitting than plain decision trees.

Cons:

- Less Interpretable: Harder to explain than a single decision tree.
- Hyperparameter Tuning: Requires careful tuning (learning rate, tree depth, etc.).
- Computationally Expensive: More trees = longer training.

3. Neural Network (with Architecture: 64→32→1)

How It Works

- Forward Propagation: Input data passes through hidden layers (64 → 32 neurons) with activation functions (e.g., ReLU).
- Non-Linear Transformations: Each layer extracts higher-level features.
- Output Layer: 1 neuron (for regression/binary classification).
- Backpropagation: Adjusts weights via gradient descent to minimize loss (e.g., cross-entropy, MSE).

Pros:

- Handles Complex Patterns: Can model intricate relationships (e.g., images, text).
- Flexible Architecture: Layers/neurons can be adjusted for the problem.

Cons:

- Data-Hungry: Requires large datasets to generalize well.
- Prone to Overfitting: Needs dropout, early stopping, or L2 regularization.
- Black-Box: Hard to interpret why predictions are made.
- Computationally Intensive: GPUs often needed for large networks.